

Laporan Tugas Besar

Teori Bahasa Formal dan Automata

Milestone 4 - Intermediate Code and Interpreter



Disusun oleh kelompok :
Kerjakan J - 5

Mahatma Brahmana	13524015
Benedict Darrel Setiawan	13524057
Raymond Jonathan Dwi Putra J	13524059
Reynard Anderson Wijaya	13524111

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

Daftar Isi

Daftar Isi.....	2
BAB 1 Landasan Teori.....	4
1.1 Intermediate Code dan Intermediate Code Generation.....	4
1.2 Intermediate Code Berbasis Stack Machine.....	4
1.3 Translasi Control Flow dengan Label dan Jump.....	5
1.4 Decorated AST sebagai Sumber Kebenaran.....	5
1.5 Arsitektur Mesin Eksekusi: Virtual Machine dan Stack.....	5
1.6 Penanganan Kerentanan pada Interpreter.....	6
BAB 2 Perancangan Program.....	7
2.1 Gambaran Umum dan Teknologi yang Digunakan.....	7
2.2 Struktur Program dan Fungsi/Kelas Utama.....	7
2.2.1 Fungsi dan Modul Utama Program pada Semantic Analysis.....	9
2.3 Alur Kerja.....	11
2.4 Rancangan Intermediate Code dan Instruction Set.....	12
2.5 Rancangan Code Generator.....	12
2.6 Rancangan Interpreter (Virtual Machine).....	14
BAB 3 Implementasi.....	16
3.1 Implementasi Alur Interpreter Bahasa Arion.....	16
3.2 Implementasi Struktur Intermediate Code.....	17
3.2.1 Implementasi Struktur Instruction.....	17
3.2.2 Implementasi Opcode.....	18
3.2.3 Implementasi OPR Code.....	19
3.3 Implementasi Code Generator.....	20
3.3.1 Implementasi Fungsi Generate.....	20
3.3.2 Implementasi Fungsi Emit Instruction.....	20
3.3.3 Implementasi Fungsi Patch Instruction.....	21
3.3.4 Implementasi Resolusi Address Variabel.....	21
3.4 Implementasi Translasi Ekspresi.....	22
3.4.1 Implementasi Translasi Literal.....	22
3.4.2 Implementasi Translasi Variabel.....	22
3.4.3 Implementasi Translasi Unary Operation.....	23
3.4.4 Implementasi Translasi Binary Operation.....	23
3.5 Implementasi Translasi Statement.....	23
3.5.1 Implementasi Translasi Assignment Statement.....	23
3.5.2 Implementasi Translasi Procedure/Function Cal.....	25
3.5.3 Implementasi Translasi Write dan Writeln.....	25
3.6 Implementasi Translasi Control Flow.....	25
3.6.1 Implementasi Translasi If dan If-Else Statement.....	26
3.6.2 Implementasi Translasi While Statement.....	27

3.6.3 Implementasi Label Handling dan Backpatching.....	27
3.7 Implementasi Interpreter dan Stack Machine.....	28
3.7.1 Implementasi Alur Eksekusi Interpreter.....	28
3.7.2 Implementasi Reset State Interpreter.....	28
3.7.3 Implementasi Pengambilan Instruksi.....	29
3.7.4 Implementasi Eksekusi Instruksi.....	29
3.7.5 Implementasi Inisialisasi Memory.....	30
3.7.6 Implementasi Operasi Stack.....	31
3.7.7 Implementasi Akses Memory.....	31
3.7.8 Implementasi Operasi OPR.....	32
3.7.9 Implementasi Validasi Runtime.....	35
3.8 Implementasi Runtime Control Flow.....	35
3.8.1 Implementasi Instruksi JMP.....	36
3.8.2 Implementasi Instruksi JPC.....	36
3.8.3 Implementasi Runtime Flow pada If-Else dan While.....	36
3.9 Implementasi Runtime Error Handling.....	36
3.9.1 Implementasi Validasi Stack.....	37
3.9.2 Implementasi Validasi Memory Address.....	38
3.9.3 Implementasi Validasi Jump Target.....	39
3.9.4 Implementasi Validasi Operasi Aritmatika.....	39
3.10 Implementasi Output Intermediate Code dan Program Output.....	41
BAB 4 Pengujian.....	42
4.1 Pengujian Masukkan dan Keluaran Program.....	42
BAB 5	
Kesimpulan dan Saran.....	49
5.1 Kesimpulan.....	49
5.2 Saran.....	49
Lampiran.....	51
Daftar Pustaka.....	52

BAB 1 Landasan Teori

1.1 Intermediate Code dan Intermediate Code Generation

Intermediate Code (kode perantara) merupakan representasi program yang berada di antara source code bahasa Arion dan proses eksekusi oleh interpreter. Setelah source code berhasil melewati tahapan lexical analysis, syntax analysis, dan semantic analysis, program akan menghasilkan Decorated Abstract Syntax Tree (Decorated AST) yang telah dilengkapi informasi semantik seperti tipe data, scope, dan referensi symbol table. Decorated AST tersebut kemudian digunakan sebagai masukan bagi modul code generator.

Tahapan penerjemahan Decorated AST menjadi instruksi perantara dikenal sebagai Intermediate Code Generation. Pada implementasi Arion, proses pembangkitan kode dilakukan dengan menelusuri struktur AST secara rekursif menggunakan pendekatan Depth-First Search (DFS). Kode dibangkitkan secara bottom-up, yaitu dengan menghasilkan instruksi untuk child node terlebih dahulu sebelum menghasilkan instruksi untuk parent node. Pendekatan ini memudahkan pembentukan instruksi ekspresi karena nilai operand telah tersedia ketika operasi utama akan dieksekusi.

Hasil dari proses ini adalah sekumpulan instruksi intermediate code yang merepresentasikan perilaku program sumber secara lengkap. Intermediate code tersebut memiliki bentuk yang lebih sederhana dibandingkan source code dan dirancang agar dapat dieksekusi secara langsung oleh interpreter atau virtual machine yang digunakan pada milestone keempat.

1.2 Intermediate Code Berbasis Stack Machine

Pada implementasi milestone keempat, intermediate code direpresentasikan dalam bentuk instruksi virtual machine berbasis stack (stack machine instruction set). Representasi ini berfungsi sebagai perantara antara source code dan proses eksekusi program. Setiap instruksi terdiri atas opcode, lexical level, dan operand yang digunakan oleh interpreter untuk menjalankan operasi tertentu.

Opcode menentukan jenis operasi yang akan dilakukan selama eksekusi program. Beberapa opcode yang digunakan antara lain:

- **LIT** untuk memasukkan literal ke stack.
- **LOD** untuk mengambil nilai variabel dari memori.
- **STO** untuk menyimpan nilai ke memori.
- **INT** untuk mengalokasikan ruang memori pada stack frame.
- **JMP** untuk melakukan lompatan tanpa syarat.
- **JPC** untuk melakukan lompatan berdasarkan kondisi.
- **CAL** untuk melakukan pemanggilan prosedur atau fungsi.
- **RET** untuk mengembalikan kontrol eksekusi dari prosedur atau fungsi yang selesai dijalankan.
- **OPR** untuk menjalankan operasi aritmatika, relasional, dan logika.

Setiap instruksi yang dihasilkan oleh code generator akan disimpan secara berurutan dalam sebuah daftar instruksi. Daftar instruksi tersebut kemudian dibaca dan dieksekusi oleh interpreter menggunakan mekanisme fetch-decode-execute. Dengan representasi ini, struktur program yang kompleks seperti ekspresi, percabangan, perulangan, dan pemanggilan prosedur dapat direpresentasikan dalam bentuk instruksi yang lebih sederhana dan mudah dijalankan oleh virtual machine.

1.3 Translasi Control Flow dengan Label dan Jump

Struktur kontrol seperti IF, IF-ELSE, WHILE, REPEAT-UNTIL, FOR, dan CASE tidak dapat direpresentasikan secara langsung dalam bentuk instruksi linier. Oleh karena itu, proses code generation menerjemahkan struktur kontrol tersebut menjadi kombinasi instruksi lompatan menggunakan opcode JMP dan JPC.

Instruksi JPC digunakan untuk melakukan lompatan ketika hasil evaluasi kondisi bernilai false, sedangkan JMP digunakan untuk melakukan lompatan tanpa syarat. Dalam banyak kasus, alamat tujuan lompatan belum diketahui ketika instruksi pertama kali dibuat. Untuk mengatasi masalah tersebut digunakan teknik backpatching.

Pada teknik ini, code generator terlebih dahulu menyimpan indeks instruksi yang masih memiliki alamat tujuan kosong. Setelah seluruh bagian statement selesai diterjemahkan dan alamat tujuan diketahui, operand pada instruksi tersebut diperbarui dengan alamat yang benar. Teknik ini digunakan pada proses translasi IF, IF-ELSE, WHILE, FOR, dan struktur kontrol lainnya.

1.4 Decorated AST sebagai Sumber Kebenaran

Decorated AST merupakan hasil akhir dari proses semantic analysis yang telah dilengkapi berbagai informasi semantik. Informasi tersebut mencakup tipe data, lexical level, indeks symbol table, serta relasi antar identifier yang ditemukan selama proses analisis.

Pada milestone keempat, Decorated AST menjadi sumber utama dalam proses code generation. Code generator tidak lagi menggunakan parse tree karena parse tree masih mengandung banyak detail sintaksis yang tidak diperlukan pada tahap pembangkitan kode. Sebaliknya, Decorated AST menyediakan representasi program yang lebih ringkas sekaligus menyimpan seluruh informasi yang dibutuhkan untuk menghasilkan instruksi virtual machine.

Sebagai contoh, ketika code generator menemukan node assignment, generator dapat langsung memperoleh informasi lokasi variabel tujuan dari symbol table dan menghasilkan instruksi STO yang sesuai. Demikian pula pada operasi aritmatika, informasi tipe data yang tersimpan pada Decorated AST membantu generator menentukan instruksi operasi yang tepat.

1.5 Arsitektur Mesin Eksekusi: Virtual Machine dan Stack

Virtual Machine (VM) merupakan komponen yang bertugas mengeksekusi intermediate code hasil code generation. VM pada milestone keempat menggunakan arsitektur berbasis stack, yaitu seluruh operasi aritmatika, logika, percabangan, serta penyimpanan variabel dilakukan melalui struktur data stack.

Interpreter menjalankan instruksi menggunakan siklus fetch-decode-execute. Pada tahap fetch, interpreter mengambil instruksi berdasarkan nilai program counter. Selanjutnya instruksi diterjemahkan pada tahap decode dan dijalankan pada tahap execute sesuai opcode yang dimiliki.

Virtual machine menyimpan instruksi program pada instruction memory dan menggunakan stack sebagai media penyimpanan data selama eksekusi berlangsung. Sebagian area stack dialokasikan sebagai memori variabel melalui instruksi INT, sedangkan sisanya digunakan sebagai operand stack untuk evaluasi ekspresi. Nilai literal dimasukkan ke stack menggunakan instruksi LIT, nilai variabel dibaca menggunakan LOD, dan hasil perhitungan disimpan menggunakan STO.

Pada implementasi milestone ini, virtual machine telah mendukung operasi aritmatika, relasional, percabangan, perulangan, akses memori, dan output program. Dukungan terhadap activation record,

prosedur, dan fungsi melalui instruksi CAL masih menjadi pengembangan lanjutan sehingga belum digunakan dalam proses eksekusi.

1.6 Penanganan Kerentanan pada Interpreter

Selain menjalankan program yang valid, interpreter juga harus mampu mendeteksi berbagai kesalahan yang mungkin terjadi pada saat runtime. Oleh karena itu, pada milestone keempat ditambahkan mekanisme pemeriksaan terhadap berbagai kondisi yang berpotensi menyebabkan kegagalan eksekusi. Beberapa kesalahan yang ditangani antara lain:

- **Stack Overflow**, yaitu ketika penggunaan stack melebihi kapasitas memori yang tersedia.
- **Stack Underflow**, yaitu ketika interpreter mencoba mengambil data dari stack yang kosong.
- **Division by Zero**, yaitu ketika operasi pembagian menggunakan penyebut bernilai nol.
- **Invalid Memory Access**, yaitu ketika program mencoba mengakses alamat memori yang tidak valid.
- **Invalid Jump Address**, yaitu ketika instruksi JMP atau JPC mengarah ke alamat instruksi yang tidak tersedia.
- **Invalid Procedure Call**, yaitu ketika pemanggilan prosedur atau fungsi tidak sesuai dengan informasi yang tersimpan pada stack frame.
- **Runtime Type Error**, yaitu ketika operasi tertentu dijalankan terhadap tipe data yang tidak sesuai.

Ketika salah satu kondisi tersebut terdeteksi, interpreter akan menghentikan eksekusi program dan menampilkan pesan kesalahan yang menjelaskan penyebab kegagalan. Dengan mekanisme ini, virtual machine dapat menjalankan program secara lebih aman dan stabil.

BAB 2 Perancangan Program

2.1 Gambaran Umum dan Teknologi yang Digunakan

Pada milestone keempat ini, program yang dikembangkan merupakan lanjutan dari lexical analyzer pada milestone pertama, syntax analyzer pada milestone kedua, serta semantic analyzer pada milestone ketiga. Setelah source code Arion berhasil melewati proses lexical analysis, syntax analysis, dan semantic analysis, program akan melanjutkan proses ke tahap code generation dan execution. Tahap ini bertujuan untuk mengubah program Arion yang telah valid secara sintaksis dan semantik menjadi representasi instruksi yang dapat dijalankan oleh mesin virtual yang dikembangkan pada milestone ini.

Secara umum, program menerima file input yang berisi source code Arion. Source code tersebut diproses oleh lexer untuk menghasilkan token, kemudian diperiksa oleh parser untuk membentuk parse tree. Parse tree kemudian dikonversi menjadi Abstract Syntax Tree (AST) dan dianalisis oleh semantic analyzer. Jika tidak ditemukan kesalahan semantik, AST akan diberikan kepada modul code generator untuk diterjemahkan menjadi intermediate code atau instruksi mesin virtual. Instruksi yang dihasilkan kemudian dieksekusi oleh interpreter sehingga program Arion dapat dijalankan dan menghasilkan output sesuai logika program yang dibuat.

Hasil utama pada milestone ini berupa daftar instruksi hasil code generation serta hasil eksekusi program oleh interpreter. Dengan adanya tahap ini, sistem compiler yang dibangun tidak hanya mampu memeriksa kebenaran program, tetapi juga dapat menjalankan program tersebut.

Program dibuat menggunakan bahasa C++ karena seluruh milestone sebelumnya juga dikembangkan menggunakan bahasa yang sama. Selain menyediakan performa yang baik, C++ memiliki dukungan struktur data seperti vector, stack, map, queue, dan class yang mempermudah implementasi lexer, parser, semantic analyzer, code generator, dan interpreter.

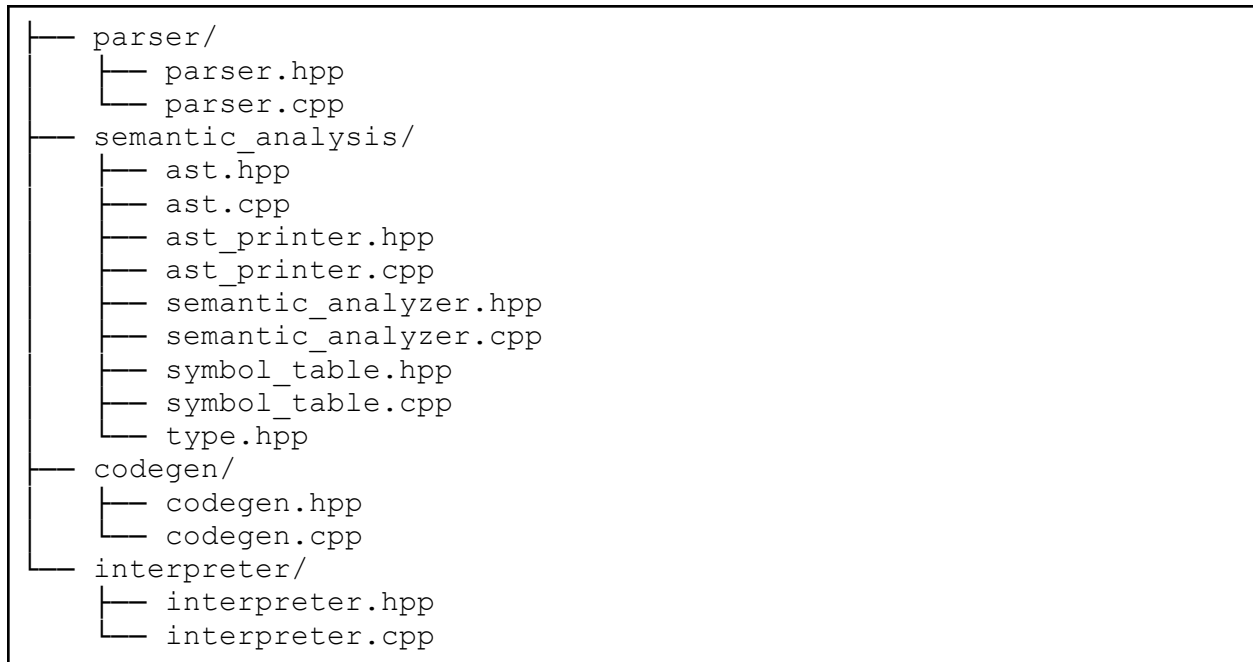
Program juga menggunakan Makefile untuk mempermudah proses kompilasi. Dengan satu perintah, seluruh modul program dapat dikompilasi dan dihubungkan sehingga proses pengujian menjadi lebih praktis.

2.2 Struktur Program dan Fungsi/Kelas Utama

Untuk menjaga keteraturan dan kemudahan pengembangan, program dirancang secara modular. Setiap tahap kompilasi dipisahkan ke dalam modul tersendiri sehingga proses lexical analysis, syntax analysis, semantic analysis, code generation, dan execution dapat dilakukan secara independen namun tetap saling terintegrasi.

Struktur direktori program secara umum adalah sebagai berikut:

```
src/  
├── main.cpp  
├── lexer/  
│   ├── lexer.hpp  
│   ├── lexer.cpp  
│   ├── token.hpp  
│   └── token.cpp
```



Penjelasan dari masing-masing bagian direktori adalah sebagai berikut:

1. **src/** : Merupakan direktori utama yang menyimpan seluruh source code program.
2. **src/main.cpp** : Merupakan file utama yang mengatur alur eksekusi program. File ini menghubungkan proses pembacaan input, tokenisasi oleh lexer, parsing oleh parser, pembentukan AST, semantic analysis, dan pencetakan hasil akhir ke terminal serta file output.
3. **src/lexer/** : Merupakan folder yang berisi bagian lexical analyzer. Bagian ini digunakan untuk membaca source code Arion dan mengubahnya menjadi daftar token yang akan digunakan oleh parser.
4. **src/parser/** : Merupakan folder yang berisi bagian syntax analyzer atau parser. Parser digunakan untuk memeriksa susunan token berdasarkan grammar bahasa Arion dan membentuk parse tree.
5. **src/semantic_analysis/** : Merupakan folder yang berisikan bagian semantic analyzer. Bagian ini digunakan untuk memeriksa apakah semantic pada program sudah sesuai atau tidak. Folder ini berisi struktur AST, proses konversi parse tree menjadi AST, semantic analyzer, symbol table, serta printer untuk menampilkan Decorated AST dan symbol table.
6. **src/codegen/** : Merupakan folder yang berisi bagian code generator. Bagian ini digunakan untuk menerjemahkan hasil semantic analysis yang telah direpresentasikan dalam bentuk Abstract Syntax Tree (AST) menjadi instruksi-instruksi mesin virtual (virtual machine instructions). Modul ini bertanggung jawab menghasilkan kode antara (intermediate code) yang dapat dieksekusi oleh interpreter. Selain itu, folder ini juga berisi definisi opcode, representasi instruksi,

serta fungsi-fungsi yang digunakan untuk membangun alur kontrol seperti assignment, operasi aritmatika, percabangan, dan perulangan.

7. **src/interpreter/** Merupakan folder yang berisi bagian interpreter atau virtual machine. Bagian ini digunakan untuk mengeksekusi instruksi yang dihasilkan oleh code generator. Interpreter bertugas mengelola memori, stack, program counter, serta menjalankan setiap instruksi sesuai opcode yang diberikan. Selain itu, modul ini juga menangani operasi aritmatika, operasi relasional, akses memori, percabangan, perulangan, serta berbagai pengecekan kesalahan saat eksekusi (runtime error) seperti pembagian dengan nol, akses memori di luar batas, stack overflow, dan stack underflow.
8. **doc/** : Merupakan folder yang digunakan untuk menyimpan laporan.
9. **test/** : Merupakan folder yang digunakan untuk menyimpan file input dan output pengujian.
10. **Makefile** : Digunakan untuk membantu proses kompilasi program agar dapat dijalankan dengan lebih mudah melalui satu perintah.

2.2.1 Fungsi dan Modul Utama Program pada Semantic Analysis

Pada milestone keempat ini, program tidak hanya melakukan lexical analysis, syntax analysis, dan semantic analysis seperti pada milestone sebelumnya, tetapi juga menambahkan tahap code generation dan execution. Oleh karena itu, terdapat beberapa modul utama yang saling terintegrasi untuk memproses source code Arion mulai dari pembacaan program hingga eksekusi instruksi pada virtual machine.

- **Modul Code Generator (codegen.hpp dan codegen.cpp) :**

Modul ini digunakan untuk menghasilkan intermediate code berupa instruksi virtual machine berdasarkan Decorated AST yang telah lolos semantic analysis. Hasil dari modul ini berupa kumpulan instruksi yang dapat dieksekusi oleh interpreter.

- **class CodeGenerator** : Kelas yang bertanggung jawab melakukan traversal terhadap Decorated AST dan menerjemahkan setiap node menjadi instruksi virtual machine.
- **generate()** : Fungsi utama untuk memulai proses code generation dari root AST. Fungsi ini menginisialisasi alokasi memori program, menghasilkan kode deklarasi, menghasilkan kode blok program utama, dan menambahkan instruksi penghentian program.
- **emit()** : Digunakan untuk menambahkan instruksi baru ke dalam daftar intermediate code.
- **patch()** : Digunakan untuk memperbaiki operand pada instruksi percabangan dan perulangan yang alamat tujuannya belum diketahui saat instruksi pertama kali dibangkitkan (backpatching).

- **genDeclarations() dan genBlock()** : Digunakan untuk menghasilkan kode deklarasi serta blok statement program.
- **genStatement()** : Digunakan untuk menerjemahkan berbagai jenis statement seperti assignment, if, while, repeat, for, case, dan procedure call.
- **genExpression()** : Digunakan untuk menghasilkan kode ekspresi secara rekursif. Fungsi ini menerjemahkan literal, variabel, operasi unary, operasi biner, akses array, serta pemanggilan fungsi.
- **genProcDecl() dan genFuncDecl()** : Digunakan untuk menghasilkan kode prosedur dan fungsi, termasuk pembentukan area memori lokal, parameter, serta instruksi pengembalian.
- **Instruction dan Opcode** : Digunakan untuk merepresentasikan instruksi virtual machine. Setiap instruksi terdiri atas nomor instruksi, opcode, lexical level, dan operand.

Instruksi yang dihasilkan mencakup operasi pemuatan literal (LIT), akses memori (LOD, STO, PLO, dan PST), operasi aritmatika dan relasional (OPR), pemanggilan subprogram (CAL), alokasi memori (INT), percabangan (JMP dan JPC), serta penghentian prosedur, fungsi, maupun program (RET).

- **Modul Interpreter (interpreter.hpp dan interpreter.cpp) :**

Modul ini digunakan untuk mengeksekusi instruksi yang dihasilkan oleh code generator. Interpreter berperan sebagai virtual machine yang menjalankan program Arion setelah seluruh proses analisis dan code generation selesai dilakukan.

- **class Interpreter** : Kelas utama yang mengelola proses eksekusi instruksi, stack, memori, program counter, serta runtime environment.
- **execute()** : Fungsi utama yang menjalankan seluruh instruksi virtual machine secara berurutan menggunakan siklus fetch-decode-execute hingga program selesai atau terjadi runtime error.
- **fetch()** : Digunakan untuk mengambil instruksi yang sedang ditunjuk oleh program counter serta memeriksa validitas alamat instruksi.
- **executeInstruction()** : Menjalankan satu instruksi virtual machine sesuai opcode yang dimiliki, seperti INT, LIT, LOD, STO, JMP, JPC, OPR, dan RET.
- **executeOpr()** : Menangani pelaksanaan operasi-operasi yang direpresentasikan melalui opcode OPR, seperti operasi aritmatika, relasional, dan output.
- **initializeMemory()** : Digunakan untuk mengalokasikan area memori awal yang diperlukan oleh program berdasarkan instruksi INT.

- **readMemory() dan writeMemory()** : Digunakan untuk membaca dan menulis nilai pada alamat memori tertentu.
- **pushValue() dan popValue()** : Digunakan untuk mengelola operand stack selama proses eksekusi berlangsung.
- **validateAddress() dan validateInstructionTarget()** : Digunakan untuk memastikan bahwa akses memori maupun tujuan lompatan instruksi berada pada rentang yang valid.
- **checkedInt32()** : Digunakan untuk mendeteksi overflow dan underflow pada operasi aritmatika sebelum hasil disimpan kembali ke stack.
- **RuntimeValue** : Struktur data yang digunakan untuk merepresentasikan nilai yang diproses oleh virtual machine selama eksekusi berlangsung.
- **RuntimeError** : Kelas exception yang digunakan untuk menangani berbagai kesalahan runtime, seperti division by zero, modulo by zero, stack overflow, stack underflow, akses memori di luar batas, overflow aritmatika, serta lompatan ke alamat instruksi yang tidak valid.

2.3 Alur Kerja

Alur kerja program dimulai dari pembacaan source code Arion. Source code diproses oleh lexer untuk menghasilkan token-token yang kemudian diberikan kepada parser. Parser melakukan pengecekan grammar dan membentuk parse tree. Parse tree selanjutnya dikonversi menjadi AST yang lebih ringkas dan sesuai untuk analisis semantik.

AST kemudian diberikan kepada semantic analyzer untuk memeriksa deklarasi identifier, scope, kompatibilitas tipe data, penggunaan array, record, fungsi, dan prosedur. Jika ditemukan semantic error, proses akan dihentikan dan error ditampilkan kepada pengguna.

Apabila semantic analysis berhasil, AST diberikan kepada code generator. Pada tahap ini setiap node AST diterjemahkan menjadi instruksi mesin virtual. Instruksi yang dihasilkan disimpan dalam urutan tertentu sehingga dapat dijalankan oleh interpreter.

Interpreter kemudian membaca instruksi satu per satu menggunakan program counter. Instruksi seperti operasi aritmatika, assignment, percabangan, perulangan, input, dan output akan dieksekusi sesuai definisi opcode yang digunakan. Selama proses ini interpreter menggunakan stack runtime dan area memori untuk menyimpan nilai yang dibutuhkan.

Setelah seluruh instruksi selesai dijalankan, interpreter menghentikan eksekusi dan menampilkan hasil akhir program kepada pengguna.

2.4 Rancangan Intermediate Code dan Instruction Set

Setelah source code Arion berhasil melewati tahap lexical analysis, syntax analysis, dan semantic analysis, program memasuki tahap code generation. Pada tahap ini, Abstract Syntax Tree (AST) yang telah divalidasi akan diterjemahkan menjadi sekumpulan instruksi intermediate yang dapat dieksekusi oleh virtual machine. Intermediate code digunakan sebagai representasi program yang lebih sederhana dibandingkan AST sehingga proses eksekusi dapat dilakukan secara lebih efisien.

Rancangan intermediate code pada milestone keempat ini menggunakan pendekatan stack-based virtual machine. Setiap operasi direpresentasikan sebagai instruksi yang bekerja terhadap operand stack. Nilai yang digunakan dalam suatu ekspresi akan terlebih dahulu dimasukkan ke stack, kemudian instruksi operasi akan mengambil operand dari stack tersebut dan mengembalikan hasilnya ke stack.

Setiap instruksi direpresentasikan oleh struktur Instruction yang terdiri atas opcode, lexical level, dan operand. Opcode menentukan operasi yang akan dilakukan oleh interpreter, sedangkan operand digunakan sebagai parameter tambahan yang diperlukan selama eksekusi. Instruction set yang digunakan dapat dikelompokkan menjadi beberapa kategori utama:

- Instruksi pemuatan data, seperti `LIT` untuk memasukkan konstanta ke stack dan `LOD` untuk mengambil nilai variabel dari memori.
- Instruksi penyimpanan data, seperti `STO` yang digunakan untuk menyimpan nilai dari stack ke memori.
- Instruksi alokasi memori, seperti `INT` yang digunakan untuk mengalokasikan ruang bagi variabel lokal pada stack frame.
- Instruksi operasi, yang direpresentasikan melalui opcode `OPR`. Berbagai operasi aritmatika, logika, dan relasional dibedakan melalui subkode operasi yang diproses oleh interpreter.
- Instruksi pengendalian alur, seperti `JMP` untuk lompatan tanpa syarat dan `JPC` untuk lompatan berdasarkan kondisi.
- Instruksi pemanggilan subprogram, seperti `CAL` untuk memanggil prosedur atau fungsi dan `RET` untuk mengembalikan kontrol ke pemanggil.

Dengan rancangan tersebut, seluruh konstruksi bahasa Arion dapat diterjemahkan menjadi urutan instruksi yang lebih sederhana dan dapat dijalankan secara langsung oleh virtual machine.

2.5 Rancangan Code Generator

Code Generator merupakan modul yang bertugas menerjemahkan Decorated AST menjadi intermediate code yang dapat dieksekusi oleh virtual machine. Modul ini diimplementasikan pada file

`codegen.hpp` dan `codegen.cpp`. Hasil keluaran dari modul ini berupa daftar instruksi virtual machine yang disimpan dalam bentuk `vector<Instruction>`.

Proses code generation dilakukan melalui traversal AST secara rekursif menggunakan pendekatan Depth-First Search (DFS). Setiap node AST memiliki aturan translasi yang berbeda sesuai dengan makna semantiknya.

Untuk node literal seperti integer, real, boolean, dan string, generator menghasilkan instruksi `LIT` untuk memasukkan nilai literal ke stack. Untuk node variabel, generator menghasilkan instruksi `LOD` untuk mengambil nilai variabel dari memori berdasarkan informasi alamat yang diperoleh dari symbol table.

Pada node assignment, generator terlebih dahulu menghasilkan instruksi untuk mengevaluasi ekspresi di sisi kanan. Setelah nilai hasil ekspresi berada di stack, instruksi `STO` dihasilkan untuk menyimpan nilai tersebut ke alamat variabel tujuan.

Untuk ekspresi aritmatika dan logika, generator menghasilkan kode operand kiri terlebih dahulu, kemudian operand kanan. Setelah kedua operand tersedia di stack, generator menghasilkan instruksi `OPR` yang sesuai. Sebagai contoh, ekspresi:

`a + b`

akan diterjemahkan menjadi:

`LOD a`

`LOD b`

`OPR ADD`

Struktur kontrol seperti `if`, `if-else`, `while`, dan `for` diterjemahkan menggunakan kombinasi instruksi `JMP` dan `JPC`. Karena alamat tujuan lompatan sering kali belum diketahui saat instruksi dibuat, generator menggunakan teknik backpatching melalui fungsi `patchJump()` untuk memperbaiki alamat tujuan setelah seluruh blok kode berhasil dibangkitkan.

Untuk prosedur dan fungsi, generator menghasilkan instruksi `CAL` yang digunakan untuk memindahkan kontrol eksekusi ke alamat subprogram yang sesuai. Sebelum instruksi `CAL` dijalankan, generator akan menghasilkan instruksi yang diperlukan untuk mempersiapkan parameter dan activation record. Setelah subprogram selesai dieksekusi, instruksi `RET` digunakan untuk mengembalikan kontrol ke pemanggil.

Dengan pendekatan ini, seluruh fitur bahasa Arion dapat diterjemahkan secara konsisten ke dalam bentuk intermediate code yang siap dieksekusi oleh virtual machine.

2.6 Rancangan Interpreter (Virtual Machine)

Interpreter merupakan komponen yang bertugas mengeksekusi intermediate code yang dihasilkan oleh code generator. Modul ini diimplementasikan pada file `interpreter.hpp` dan `interpreter.cpp`. Interpreter berfungsi sebagai virtual machine yang mensimulasikan lingkungan eksekusi program Arion.

Virtual machine menggunakan arsitektur berbasis stack dan memiliki beberapa komponen utama, yaitu:

- Instruction Memory, yang menyimpan daftar instruksi hasil code generation.
- Operand Stack, yang digunakan untuk evaluasi ekspresi dan penyimpanan sementara nilai selama eksekusi berlangsung.
- Runtime Memory, yang digunakan untuk menyimpan nilai variabel program. Pada implementasi ini, area memori dialokasikan melalui instruksi INT dan disimpan pada bagian awal stack.
- Program Counter (PC), yang menunjukkan alamat instruksi yang sedang dieksekusi.

Proses eksekusi dilakukan menggunakan siklus fetch-decode-execute. Pada tahap fetch, interpreter mengambil instruksi yang ditunjuk oleh Program Counter. Selanjutnya instruksi diterjemahkan pada tahap decode. Setelah itu instruksi dijalankan pada tahap execute sesuai opcode yang dimiliki.

Untuk instruksi `LIT`, interpreter memasukkan nilai literal ke operand stack. Instruksi `LOD` digunakan untuk mengambil nilai dari alamat memori tertentu dan memasukkannya ke stack. Sebaliknya, instruksi `STO` digunakan untuk mengambil nilai dari stack dan menyimpannya ke alamat memori yang ditentukan.

Instruksi `OPR` digunakan untuk menjalankan berbagai operasi aritmatika, relasional, dan output program. Operasi yang didukung meliputi negasi, penjumlahan, pengurangan, perkalian, pembagian, modulo, perbandingan nilai, serta pencetakan output melalui operasi `WRT` dan `WRTLN`. Seluruh operand operasi diambil dari stack dan hasil operasi akan dikembalikan ke stack.

Instruksi `JMP` dan `JPC` digunakan untuk mengendalikan alur eksekusi program. `JMP` memindahkan Program Counter secara langsung ke alamat tujuan, sedangkan `JPC` melakukan lompatan hanya apabila kondisi yang berada pada puncak stack bernilai false. Mekanisme ini digunakan untuk merepresentasikan struktur kontrol seperti percabangan dan perulangan.

Interpreter juga mendukung instruksi `PLO` dan `PST` untuk melakukan akses memori tidak langsung (pointer-based access). Instruksi ini memungkinkan program membaca atau menulis nilai berdasarkan alamat yang diperoleh saat runtime.

Selain menjalankan program, interpreter menangani berbagai runtime error yang dapat terjadi selama eksekusi. Kesalahan yang diperiksa meliputi pembagian dengan nol, modulo dengan nol, stack overflow, stack underflow, overflow dan underflow aritmatika, akses memori di luar batas yang dialokasikan, alamat lompatan yang tidak valid, serta penggunaan opcode yang tidak dikenali. Penanganan kesalahan dilakukan melalui mekanisme `RuntimeError` sehingga eksekusi program dapat dihentikan secara aman dan pesan kesalahan dapat ditampilkan kepada pengguna.

Pada implementasi milestone keempat ini, instruction set telah menyediakan opcode `CAL` untuk pemanggilan prosedur atau fungsi. Namun mekanisme eksekusi instruksi tersebut masih belum diimplementasikan pada interpreter sehingga fitur prosedur dan fungsi belum digunakan dalam proses eksekusi program.

Dengan rancangan ini, virtual machine mampu mengeksekusi intermediate code yang dihasilkan oleh code generator untuk mendukung evaluasi ekspresi, manipulasi variabel, percabangan, perulangan, dan output program secara langsung pada milestone keempat.

BAB 3 Implementasi

3.1 Implementasi Alur Interpreter Bahasa Arion

Program Arion pada tugas besar ini dirancang sebagai interpreter bahasa pemrograman sederhana. Interpreter tidak langsung menjalankan source code secara langsung, melainkan memproses source code melalui beberapa tahap terlebih dahulu hingga menjadi representasi yang lebih mudah dieksekusi. Tahap-tahap tersebut terdiri dari pembacaan source code, tokenisasi, parsing, pembentukan AST, analisis semantik, pembentukan intermediate code, dan eksekusi intermediate code. Secara umum, interpreter bahasa Arion bekerja dengan memisahkan proses front-end dan back-end. Bagian front-end bertugas memahami source code yang ditulis pengguna. Source code dibaca dari file input, kemudian diproses oleh lexer untuk menghasilkan token, yang dilanjutkan dengan pembentukan parse tree oleh parser. Setelah parser tree terbentuk, parse tree kemudian diubah menjadi AST melalui proses pembentukan node-node yang lebih ringkas oleh semantic analyzer. Proses ini telah dilakukan pada milestone tugas besar sebelumnya.

Bagian back-end berfokus pada eksekusi program yang dilanjutkan pada milestone empat ini. AST yang sudah valid secara semantik tidak langsung dieksekusi sebagai struktur pohon, tetapi diterjemahkan terlebih dahulu menjadi intermediate code yang berbentuk instruksi linear. Instruksi linear tersebut lebih mudah dijalankan oleh interpreter dengan membaca instruksi satu per satu menggunakan program counter. Apabila pada proses sebelumnya tidak ada terjadi masalah, seperti syntax error ataupun semantic error, AST yang sudah didekorasi digunakan sebagai masukan untuk CodeGenerator. Hasil ini CodeGenerator berupa daftar instruksi intermediate code kemudian diberikan kepada Interpreter untuk dieksekusi model stack machine. Program utama juga bertanggung jawab untuk menyatukan seluruh output yang dilakukan pada masing masing tugas yang ada. Output program tidak hanya berisi Decorated AST dan symbol table, tetapi juga intermediate code dan output aktual dari eksekusi program Arion.

```
vector<Token> tokens = tokenize(inputFile);
Parser parser(tokens);
TreeParser* parseTree = parser.parse();
ASTNode* ast = buildAST(parseTree);
SemanticAnalyzer analyzer;
analyzer.analyze(ast);

if (!analyzer.hasErrors()) {
    CodeGenerator generator;
    generator.generate(ast);

    std::ostringstream runtimeStream;
    Interpreter interpreter;
```

```

    interpreter.execute(generator.getInstructions(),
runtimeStream);
}

```

Komponen	Kode	Keterangan
Lexer	tokenize(inputFile)	Mengubah source code menjadi token
Parser	Parser::parser()	Membentuk parse tree dari token
AST builder	buildAST(parseeTree)	Membentuk AST dari parse tree
Semantic analyzer	analyzer.analyze(ast)	Menvalidasi AST secara semantik
Code generator	generator.generate(ast)	Membentuk intermediate code
Interpreter	interpreter.execute(...)	Mengeksekusi intermediate code

3.2 Implementasi Struktur Intermediate Code

Intermediate code digunakan sebagai representasi perantara antara Decoreated AST dan interpreter. Sturktur ini dibuat agar program Arion dapat dieksekusi sebagai intruksi linear berbasis stack machine. Setiap instruksi memiliki opcode dan operand sehingga intepreter dapat mengeksekusi program tanpa harus menelusuri AST kembali.

3.2.1 Implementasi Struktur Instruction

Struktur `Instruction` merepresentasikan satu baris intermediate code. Setiap instruksi menyimpan nomor baris, opcode, lexical level, dan operand. Nomor baris digunakan untuk menunjukkan posisi instruksi dalam daftar intermediate code. Opcode digunakan untuk menentukan jenis instruksi yang dijalankan. Operand digunakan untuk menyimpan nilai literal, alamar variabel, nomor operasi, atau target jump.

```

struct Instruction {
    int    line;    // nomor baris
    Opcode opcode;  // jenis instruksi
    int    level;   // lexical level (biasanya 0)
    int    operand; // nilai operand
};

```

Komponen	Kode	Keterangan
Nomor instruksi	line	Menunjukkan posisi intruksi
Opcode	opcode	Menentukan jenis instruksi
Lexical level	level	Menyimpan level
Operand	operand	Menyimpan data tambahan instruksi

3.2.2 Implementasi Opcode

Opcode adalah daftar instruksi utama yang dapat dieksekusi oleh interpreter. Opcode ini mengikuti model stack machine, seperti load literal, load value, store value, inisialisasi memori, jump, operation, dan return.

```
enum class Opcode {
    LIT, // Load Literal
    LOD, // Load Value dari address
    STO, // Store Value ke address
    PLO, // Load Value ke address yang di pop dari stack
    PST, // Store Value ke address yang di pop dari stack
    CAL, // Call fungsi/prosedur
    INT, // Initiate Memory
    JMP, // Unconditional Jump
    JPC, // Conditional Jump (jump if false)
    OPR, // Operation
    RET  // Return
};
```

Instruksi	Kode	Keterangan
Load literal	LIT	Memasukkan literal ke stack
Load value	LOD	Mengambil nilai dari address memori
Store value	STO	Menyimpan memori runtime
Load Pointer Value	PLO	Mengambil nilai dari address memori yang di pop dari stack

Store Pointer Value	PST	Menyimpan nilai dari address memori yang di pop dari stack
Jump	JMP	Melompat tanpa kondisi
Conditional jump	JPC	Melompat jika kondisi false
Operation	OPR	Menjalankan operasi tertentu
Return	RET	Mengakhiri eksekusi

3.2.3 Implementasi OPR Code

OprCode digunakan sebagai operand dari instruksi OPR. Dengan pemisahan ini, opcode utama tetap sederhana, sedangkan operasi seperti aritmatika, relasional, dan output dikelompokkan ke dalam nomor operasi.

```
enum class OprCode {
    NEG    = 1,
    ADD    = 2,
    SUB    = 3,
    MUL    = 4,
    DIV    = 5,
    MOD    = 6,
    EQL    = 7,
    NEQ    = 8,
    LSS    = 9,
    GEQ    = 10,
    GTR    = 11,
    LEQ    = 12,
    WRT    = 13,
    WRTLN  = 14
};
```

Operasi	Kode	Keterangan
Aritmatika	ADD, SUB, MUL, DIV, MOD	Mengolah dua operand numerik
Unary	NEG	Melakukan negasi nilai
Relasional	EQL, NEQ, LSS, GEQ, GTR, LEQ	Menghasilkan nilai boolean
Output	WRT, WRTLN	Mencetak nilai runtime

3.3 Implementasi Code Generator

Code generator diimplementasikan melalui kelas `CodeGenerator`. Kelas ini menerima Decorated AST dan menghasilkan daftar Instruction. Generator bekerja dengan menelusuri AST, mengubah ekspresi menjadi operasi stack, mengubah statement menjadi instruksi linear, serta mengisi target jump menggunakan mekanisme patching.

3.3.1 Implementasi Fungsi Generate

Fungsi `generate()` menjadi titik masuk proses pembentukan intermediate code. Fungsi ini memvalidasi root AST sebagai `ProgramNode`, mengambil bagian deklarasi dan block, menghitung kebutuhan memori, membentuk intruksi INT, lalu menghasilkan instruksi untuk deklarasi dan statement utama.

```
void CodeGenerator::generate(ASTNode* root) {
    if (!root) return;

    ProgramNode* program = dynamic_cast<ProgramNode*>(root);
    if (!program) {
        std::cerr << "[CodeGen] Root bukan ProgramNode\n";
        return;
    }

    auto ch = program->getChildren();
    ProcCallNode* decls = dynamic_cast<ProcCallNode*>(ch[0]);
    ProcCallNode* block = dynamic_cast<ProcCallNode*>(ch[1]);

    int varCount = decls ? countVars(decls) : 0;
    int memSize = 3 + varCount;

    emit(Opcode::INT, 0, memSize);
    if (decls) genDeclarations(decls);
    if (block) genBlock(block);
    emit(Opcode::RET, 0, 0);
}
```

3.3.2 Implementasi Fungsi Emit Instruction

Fungsi `Emit()` digunakan untuk menambahkan intruksi baru ke vector instructions. Nomor baris instruksi ditentukan dari ukuran vector saat instruksi dibuat.

```
int CodeGenerator::emit(Opcode op, int level, int operand) {
    int line = static_cast<int>(instructions.size());
    instructions.push_back({line, op, level, operand});
    return line;
}
```

3.3.3 Implementasi Fungsi Patch Instruction

Fungsi `patch()` digunakan untuk memperbaiki operand instruksi yang belum diketahui targetnya ketika pertama kali dibuat. Mekanisme ini penting untuk translasi control flow seperti `if`, `while`, dan `case`.

```
void CodeGenerator::patch(int instrIndex, int newOperand) {
    if (instrIndex >= 0 && instrIndex <
        static_cast<int>(instructions.size()))
        instructions[instrIndex].operand = newOperand;
}
```

3.3.4 Implementasi Resolusi Address Variabel

Resolusi address variabel dilakukan dengan menyimpan pasangan nama variabel dan alamat memori. Jika variabel sudah pernah dialokasikan, fungsi mengembalikan alamat yang sama. Jika belum, variabel diberi address baru.

```
int CodeGenerator::resolveAddress(const std::string& name) {
    std::string key = lower(name);
    auto it = varAddress.find(key);
    if (it != varAddress.end()) return it->second;
    int addr = nextAddress++;
    varAddress[key] = addr;
    return addr;
}
```

3.4 Implementasi Translasi Ekspresi

Translasi ekspresi dilakukan dengan menelusuri node ekspresi dan membangkitkan instruksi stack machine. Operand dibangkitkan terlebih dahulu, lalu operasi dijalankan menggunakan OPR.

3.4.1 Implementasi Translasi Literal

Literal angka diterjemahkan menjadi instruksi LIT. Untuk integer, nilai langsung diambil menggunakan `stoi`. Untuk real, nilai dibaca menggunakan `stod` lalu dikonversi menjadi representasi floating point 64 bit di runtime saat ini.

```
void CodeGenerator::genNumber(NumberNode* node) {
    try {

        if (node->num.find('.') == std::string::npos) {
            int val = std::stoi(node->num);
            emit(Opcode::LIT, 0, wrapRuntimeType(RuntimeType::INT,
val));
        } else {
            double val = std::stod(node->num);
            float f = static_cast<float>(val);
            uint64_t bits;
            std::memcpy(&bits, &f, sizeof(f));
            emit(Opcode::LIT, 0, wrapRuntimeType(RuntimeType::REAL,
static_cast<int>(bits)));
        }
    } catch (...) {
        emit(Opcode::LIT, 0, 0);
    }
}
```

3.4.2 Implementasi Translasi Variabel

Variabel diterjemahkan menjadi instruksi LOD. Address variabel dicari melalui `resolveAddress()`, lalu nilai pada address tersebut akan dimuat ke stack saat runtime.

```
void CodeGenerator::genUnaryOp(UnaryOpNode* node) {
    genExpression(node->value);
    std::string op = lower(node->op);
    if (op == "minus" || op == "-") {
        emit(Opcode::OPR, 0, static_cast<int>(OprCode::NEG));
    } else if (op == "not") {
        emit(Opcode::LIT, 0, 1);
        emit(Opcode::OPR, 0, static_cast<int>(OprCode::EQL));
    }
}
```

```

    }
}

```

3.4.3 Implementasi Translasi Unary Operation

Unary operation diproses setelah operand dibangkitkan. Unary minus diterjemahkan menjadi OPR NEG. Operator not diterjemahkan menjadi perbandingan dengan literal 1.

```

void CodeGenerator::genUnaryOp(UnaryOpNode* node) {
    genExpression(node->value);
    std::string op = lower(node->op);
    if (op == "minus" || op == "-") {
        emit(Opcode::OPR, 0, static_cast<int>(OprCode::NEG));
    } else if (op == "not") {
        emit(Opcode::LIT, 0, 1);
        emit(Opcode::OPR, 0, static_cast<int>(OprCode::EQL));
    }
}

```

3.4.4 Implementasi Translasi Binary Operation

Binary operation membangkitkan operand kiri dan kanan terlebih dahulu. Setelah kedua operand berada di stack, generator membentuk instruksi OPR sesuai operator yang digunakan.

```

void CodeGenerator::genBinOp(BinOpNode* node) {
    genExpression(node->left);
    genExpression(node->right);
    OprCode opr = binopToOpr(node->op);
    emit(Opcode::OPR, 0, static_cast<int>(opr));
}

```

3.5 Implementasi Translasi Statement

Statement diterjemahkan berdasarkan jenis node AST. Code generator memilih fungsi translasi yang sesuai, seperti assignment, procedure call, control flow, loop, dan case.

3.5.1 Implementasi Translasi Assignment Statement

Assignment diterjemahkan dengan membangkitkan ekspresi nilai terlebih dahulu, lalu menyimpan hasilnya ke address target menggunakan STO. Untuk penyimpanan ke sebuah elemen array, pertama dilakukan operasi di runtime untuk mendapatkan offset dari address berdasarkan indeks array, lalu offset tersebut dijumlahkan dengan address dari indeks awal, dan akhirnya memanggil instruksi PST

untuk penyimpanan nilai di address tersebut. Untuk penyimpanan ke sebuah field record pertama dilakukan translasi dari field name menjadi offset yang akan dijumlahkan dengan address dari variabel, dan akhirnya memanggil instruksi PST untuk penyimpanan nilai di address tersebut.

```
void CodeGenerator::genAssign(AssignNode* node) {
    if (!node) return;

    genExpression(node->value);

    VarNode* targetVar = dynamic_cast<VarNode*>(node->target);
    if (targetVar) {
        int addr = resolveAddress(targetVar->name);
        emit(Opcode::STO, 0, addr);
    } else if (ArrayAccessNode* targetVar =
dynamic_cast<ArrayAccessNode*>(node->target)) {
        int addr = resolveAddress(targetVar->name);
        genExpression(targetVar->idx);

        int inset =
table->getATab().at(table->lookup(targetVar->name)->ref).low;

        emit(Opcode::LIT, 0, wrapRuntimeType(RuntimeType::INT,
inset));

        emit(Opcode::OPR, 0, static_cast<int>(OprCode::SUB));

        emit(Opcode::LIT, 0, wrapRuntimeType(RuntimeType::INT,
addr));

        emit(Opcode::OPR, 0, static_cast<int>(OprCode::ADD));

        emit(Opcode::PST, 0, 0);
    } else if (RecordAccessNode* targetVar =
dynamic_cast<RecordAccessNode*>(node->target)) {
        int addr = resolveAddress(targetVar->name);

        emit(Opcode::LIT, 0, wrapRuntimeType(RuntimeType::INT,
fieldOffset[recordRecord[lower(targetVar->name)][targetVar->fieldN
ame]]));

        emit(Opcode::LIT, 0, wrapRuntimeType(RuntimeType::INT,
addr));

        emit(Opcode::OPR, 0, static_cast<int>(OprCode::ADD));

        emit(Opcode::PST, 0, 0);
    }
}
```

3.5.2 Implementasi Translasi Procedure/Function Cal

Procedure dan function call diterjemahkan dengan membangkitkan seluruh argumen terlebih dahulu. Jika subprogram ditemukan pada tabel alamat subprogram, generator membentuk instruksi CAL.

```
auto it = subprogramAddress.find(name);
if (it != subprogramAddress.end()) {
    for (ValueNode* arg : node->args) {
        genExpression(arg);
    }
    emit(Opcode::CAL, 0, it->second);
} else {
    std::cerr << "[CodeGen] Warning: prosedur '" << node->name <<
    "'" tidak ditemukan\n";
}
```

3.5.3 Implementasi Translasi Write dan Writeln

write dan writeln diperlakukan sebagai procedure bawaan. Generator tidak membentuk CAL, tetapi langsung membentuk operasi output melalui OPR WRT atau OPR WRTLN.

```
if (name == "writeln") {
    for (ValueNode* arg : node->args) {
        genExpression(arg);
        emit(Opcode::OPR, 0, static_cast<int>(OprCode::WRTLN));
    }
    return;
}

if (name == "write") {
    for (ValueNode* arg : node->args) {
        genExpression(arg);
        emit(Opcode::OPR, 0, static_cast<int>(OprCode::WRT));
    }
    return;
}
```

3.6 Implementasi Translasi Control Flow

Translasi control flow dilakukan pada modul CodeGenerator untuk mengubah struktur kontrol pada AST menjadi intermediate code linear. Struktur seperti if, if-else, dan while diterjemahkan menggunakan instruksi JPC dan JMP. Instruksi JPC digunakan untuk melompat ketika kondisi bernilai

false, sedangkan JMP digunakan untuk melompat tanpa syarat. Target lompatan disimpan sebagai indeks instruksi pada vector instructions.

```
void CodeGenerator::genStatement(ASTNode* node) {
    if (!node) return;

    if (AssignNode* n = dynamic_cast<AssignNode*>(node)) {
        genAssign(n); return; }
    if (IfNode* n = dynamic_cast<IfNode*>(node)) {
        genIf(n); return; }
    if (WhileNode* n = dynamic_cast<WhileNode*>(node)) {
        genWhile(n); return; }
    if (RepeatNode* n = dynamic_cast<RepeatNode*>(node)) {
        genRepeat(n); return; }
    if (ForNode* n = dynamic_cast<ForNode*>(node)) {
        genFor(n); return; }
    if (CaseNode* n = dynamic_cast<CaseNode*>(node)) {
        genCase(n); return; }
    if (ProcCallNode* n = dynamic_cast<ProcCallNode*>(node)) {
        if (n->name == "__block__") { genBlock(n);
        return; }
        if (n->name == "__declarations__") { genDeclarations(n);
        return; }
        genProcCall(n);
        return;
    }
}
```

3.6.1 Implementasi Translasi If dan If-Else Statement

Translasi if dan if-else dilakukan menggunakan fungsi `genIf()`. Fungsi ini membangkitkan kondisi, membuat JPC untuk percabangan false, dan menggunakan JMP jika terdapat blok else.

```
void CodeGenerator::genIf(IfNode* node) {
    if (!node) return;

    genExpression(node->condition);

    int jpcIdx = emit(Opcode::JPC, 0, 0);

    genStatement(node->then);

    if (node->elseThen) {
        int jmpIdx = emit(Opcode::JMP, 0, 0);
        patch(jpcIdx, nextLine());
        genStatement(node->elseThen);
        patch(jmpIdx, nextLine());
    } else {
        patch(jpcIdx, nextLine());
    }
}
```

```

    }
}

```

3.6.2 Implementasi Translasi While Statement

Translasi while dilakukan menggunakan fungsi `genWhile()`. Fungsi ini menyimpan posisi awal loop, membangkitkan kondisi, membuat `JPC` untuk keluar dari loop, dan membuat `JMP` untuk kembali ke awal loop.

```

void CodeGenerator::genWhile(WhileNode* node) {
    if (!node) return;

    int loopStart = nextLine();

    genExpression(node->condition);

    int jpcIdx = emit(Opcode::JPC, 0, 0);

    genStatement(node->statement);

    emit(Opcode::JMP, 0, loopStart);

    patch(jpcIdx, nextLine());
}

```

3.6.3 Implementasi Label Handling dan Backpatching

Label handling dilakukan menggunakan `emit()`, `nextLine()`, dan `patch()`. Fungsi `emit()` digunakan untuk menambahkan instruksi baru ke daftar intermediate code, `nextLine()` digunakan untuk mengambil indeks instruksi berikutnya sebagai target lompatan, sedangkan `patch()` digunakan untuk memperbaiki operand jump setelah posisi tujuan diketahui.

```

int CodeGenerator::emit(Opcode op, int level, int operand) {
    int line = static_cast<int>(instructions.size());
    instructions.push_back({line, op, level, operand});
    return line;
}

void CodeGenerator::patch(int instrIndex, int newOperand) {
    if (instrIndex >= 0 && instrIndex <
        static_cast<int>(instructions.size()))
        instructions[instrIndex].operand = newOperand;
}

int CodeGenerator::nextLine() const {
    return static_cast<int>(instructions.size());
}

```

```
}
```

3.7 Implementasi Interpreter dan Stack Machine

Interpreter menjalankan intermediate code dengan model stack machine. Runtime menyimpan program counter, stack, ukuran memori, jumlah langkah eksekusi, dan status halted.

3.7.1 Implementasi Alur Eksekusi Interpreter

Alur eksekusi dilakukan pada fungsi `execute()`. Fungsi ini melakukan reset state, lalu menjalankan instruksi selama program belum berhenti dan pc masih valid. Setiap instruksi diambil, pc dinaikkan, lalu instruksi dieksekusi.

```
void Interpreter::execute(const std::vector<Instruction>&
instructions,
                        std::ostream& out) {
    reset();

    while (!halted && pc < instructions.size()) {
        if (steps >= config.maxSteps) {
            throw RuntimeError(
                "Runtime Error: maximum execution step limit
exceeded"
            );
        }

        const Instruction& instruction = fetch(instructions);
        pc++;
        steps++;
        executeInstruction(instruction, instructions, out);
    }
}
```

3.7.2 Implementasi Reset State Interpreter

Reset state dilakukan sebelum program dijalankan. Stack dikosongkan, ukuran memory diatur ulang, pc dikembalikan ke 0, jumlah step direset, dan status halted dibuat false.

```
void Interpreter::reset() {
    stack.clear();
    memorySize = 0;
    pc = 0;
    steps = 0;
    halted = false;
}
```

3.7.3 Implementasi Pengambilan Instruksi

Instruksi diambil menggunakan fungsi `fetch()`. Fungsi ini membaca instruksi berdasarkan nilai `pc` dan memastikan `pc` tidak melewati batas daftar instruksi.

```
const Instruction& Interpreter::fetch(  
    const std::vector<Instruction>& instructions  
) const {  
    if (pc >= instructions.size()) {  
        throw RuntimeError("Runtime Error: instruction pointer out  
of bounds");  
    }  
    return instructions[pc];  
}
```

3.7.4 Implementasi Eksekusi Instruksi

Eksekusi instruksi dilakukan pada fungsi `executeInstruction()`. Fungsi ini membaca opcode, lalu menjalankan aksi sesuai instruksi seperti INT, LIT, LOD, STO, PLO, PST, OPR, dan RET.

```
void Interpreter::executeInstruction(  
    const Instruction& instruction,  
    const std::vector<Instruction>& instructions,  
    std::ostream& out  
) {  
    (void)instructions;  
    (void)out;  
    int ptrAddr;  
    switch (instruction.opcode) {  
        case Opcode::INT:  
            initializeMemory(instruction.operand);  
            break;  
        case Opcode::LIT:  
            pushValue(RuntimeValue(unwrapRuntimeType(instruction.operand)));  
            break;  
        case Opcode::LOD:  
            pushValue(readMemory(instruction.operand));  
            break;  
        case Opcode::STO:  
            writeMemory(instruction.operand, popValue());  
            break;  
        case Opcode::PLO:  
            ptrAddr = popValue().integer;  
            pushValue(readMemory(ptrAddr + instruction.operand));  
            break;  
        case Opcode::PST:  
            ptrAddr = popValue().integer;  
            writeMemory(ptrAddr, popValue());  
    }
```

```

        break;
    case Opcode::OPR:
        executeOpr(instruction.operand, out);
        break;
    case Opcode::RET:
        halted = true;
        break;
    case Opcode::JMP:
        validateInstructionTarget(instruction, instructions);
        pc = static_cast<std::size_t>(instruction.operand);
        break;
    case Opcode::JPC: {
        validateInstructionTarget(instruction, instructions);
        RuntimeValue condition = popValue();
        if (condition.integer == 0) {
            pc = static_cast<std::size_t>(instruction.operand);
        }
        break;
    }
    case Opcode::CAL:
        validateInstructionTarget(instruction, instructions);
        throw RuntimeError(
            "Runtime Error: instruction " +
            CodeGenerator::opcodeToString(instruction.opcode) +
            " is not supported in Orang 2 interpreter core"
        );
    default:
        throw RuntimeError(
            "Runtime Error: invalid opcode at line " +
            std::to_string(instruction.line)
        );
}
}

```

3.7.5 Implementasi Inisialisasi Memory

Memory diinisialisasi melalui fungsi `initializeMemory()`. Ukuran memory diambil dari operand instruksi INT, lalu stack diisi dengan nilai default sebanyak ukuran tersebut.

```

void Interpreter::initializeMemory(int size) {
    if (size < 0) {
        throw RuntimeError("Runtime Error: negative memory size");
    }

    if (static_cast<std::size_t>(size) > config.maxStackSize) {
        throw RuntimeError(
            "Runtime Error: Stack Overflow (initial memory exceeds
limit " +
            std::to_string(config.maxStackSize) + ")")
    }
}

```

```

    );
}

stack.assign(static_cast<std::size_t>(size), RuntimeValue());
memorySize = static_cast<std::size_t>(size);
}

```

3.7.6 Implementasi Operasi Stack

Operasi stack dilakukan melalui `pushValue()` dan `popValue()`. `pushValue()` menambahkan nilai ke stack, sedangkan `popValue()` mengambil nilai teratas tanpa mengambil bagian memory.

```

void Interpreter::pushValue(RuntimeValue value) {
    if (stack.size() >= config.maxStackSize) {
        throw RuntimeError(
            "Runtime Error: Stack Overflow (limit " +
            std::to_string(config.maxStackSize) + ")"
        );
    }
    stack.push_back(value);
}

RuntimeValue Interpreter::popValue() {
    if (stack.size() <= memorySize) {
        throw RuntimeError("Runtime Error: Stack Underflow");
    }
    RuntimeValue value = stack.back();
    stack.pop_back();
    return value;
}

```

3.7.7 Implementasi Akses Memory

Akses memory dilakukan melalui `readMemory()` dan `writeMemory()`. `readMemory()` membaca nilai dari alamat tertentu, sedangkan `writeMemory()` menyimpan nilai ke alamat tertentu setelah alamat divalidasi.

```

RuntimeValue Interpreter::readMemory(int address) const {
    validateAddress(address);
    return stack[static_cast<std::size_t>(address)];
}

void Interpreter::writeMemory(int address, RuntimeValue value) {
    validateAddress(address);
    stack[static_cast<std::size_t>(address)] = value;
}

```

3.7.8 Implementasi Operasi OPR

Instruksi OPR diproses melalui fungsi `executeOpr()`. Fungsi ini menangani operasi unary, aritmatika, perbandingan, dan output seperti WRT serta WRTLN.

```
void Interpreter::executeOpr(int operation, std::ostream& out) {
    (void)out;

    switch (static_cast<OprCode>(operation)) {
        case OprCode::NEG: {
            RuntimeValue value = popValue();
            pushValue(RuntimeValue({value.type, checkedInt32(
                -static_cast<std::int64_t>(value.integer),
                "NEG"
            ))));
            break;
        }
        case OprCode::ADD:
        case OprCode::SUB:
        case OprCode::MUL:
        case OprCode::DIV:
        case OprCode::MOD: {
            RuntimeValue right = popValue();
            RuntimeValue left = popValue();
            std::int64_t result = 0;
            RuntimeType resultType = resolveRuntimeType(left,
right);
            if (resultType == RuntimeType::ERROR) {
                throw RuntimeError("Runtime Error: Unable to do
operation with Runtime Type " +
                                runtimeTypeToString(left.type)
+ " and " + runtimeTypeToString(right.type)
                                );
            }
            std::string operationName;

            if (static_cast<OprCode>(operation) == OprCode::ADD) {
                if (resultType == RuntimeType::REAL) {
                    result = handleFloatingPointOperation(left,
right, OprCode::ADD);
                } else {
                    result =
static_cast<std::int64_t>(left.integer) + right.integer;
                }
                operationName = "ADD";
            } else if (static_cast<OprCode>(operation) ==
OprCode::SUB) {
                if (resultType == RuntimeType::REAL) {
                    result = handleFloatingPointOperation(left,
right, OprCode::SUB);
                } else {
```

```

        result =
static_cast<std::int64_t>(left.integer) - right.integer;
    }
    operationName = "SUB";
} else if (static_cast<OprCode>(operation) ==
OprCode::MUL) {
    if (resultType == RuntimeType::REAL) {
        result = handleFloatingPointOperation(left,
right, OprCode::MUL);
    } else {
        result =
static_cast<std::int64_t>(left.integer) * right.integer;
    }
    operationName = "MUL";
} else if (static_cast<OprCode>(operation) ==
OprCode::DIV) {
    if (right.integer == 0) {
        throw RuntimeError("Runtime Error: Division by
zero");
    }
    if (resultType == RuntimeType::REAL) {
        result = handleFloatingPointOperation(left,
right, OprCode::DIV);
    } else {
        result =
static_cast<std::int64_t>(left.integer) / right.integer;
    }
    operationName = "DIV";
} else {
    if (right.integer == 0) {
        throw RuntimeError("Runtime Error: Modulo by
zero");
    }
    if (resultType == RuntimeType::REAL) {
        throw RuntimeError(
            "Runtime Error: Unsupported operation for
Floating Point type"
        );
    } else {
        result =
static_cast<std::int64_t>(left.integer) % right.integer;
    }
    operationName = "MOD";
}

    pushValue(RuntimeValue({resultType,
checkedInt32(result, operationName)}));
    break;
}
case OprCode::EQL:
case OprCode::NEQ:

```

```

        case OprCode::LSS:
        case OprCode::GEQ:
        case OprCode::GTR:
        case OprCode::LEQ: {
            RuntimeValue right = popValue();
            RuntimeValue left = popValue();
            bool result = false;
            RuntimeType resultType = resolveRuntimeType(left,
right);
            if (resultType == RuntimeType::ERROR) {
                throw RuntimeError("Runtime Error: Unable to do
operation with Runtime Type " +
                                runtimeTypeToString(left.type)
+ " and " + runtimeTypeToString(right.type)
                                );
            }

            if (static_cast<OprCode>(operation) == OprCode::EQL) {
                result = left.integer == right.integer;
            } else if (static_cast<OprCode>(operation) ==
OprCode::NEQ) {
                result = left.integer != right.integer;
            } else if (static_cast<OprCode>(operation) ==
OprCode::LSS) {
                result = left.integer < right.integer;
            } else if (static_cast<OprCode>(operation) ==
OprCode::GEQ) {
                result = left.integer >= right.integer;
            } else if (static_cast<OprCode>(operation) ==
OprCode::GTR) {
                result = left.integer > right.integer;
            } else {
                result = left.integer <= right.integer;
            }

            pushValue(RuntimeValue({RuntimeType::BOOLEAN, result ?
1 : 0}));
            break;
        }
        case OprCode::WRT: {
            RuntimeValue value = popValue();
            out << resolveWriteValue(value);
            break;
        }
        case OprCode::WRTLN: {
            RuntimeValue value = popValue();
            out << resolveWriteValue(value) << '\n';
            break;
        }
        default:
            throw RuntimeError(

```

```

        "Runtime Error: invalid OPR code " +
        std::to_string(operation)
    );
}

```

3.7.9 Implementasi Validasi Runtime

Validasi runtime digunakan untuk mencegah akses atau operasi yang tidak valid. Validasi mencakup alamat memory, batas stack, target instruksi, dan operasi aritmatika tertentu.

```

void Interpreter::validateAddress(int address) const {
    if (address < 0 || static_cast<std::size_t>(address) >=
memorySize) {
        throw RuntimeError(
            "Runtime Error: Memory Access Out of Bounds at address
" +
            std::to_string(address)
        );
    }
}

```

3.8 Implementasi Runtime Control Flow

Runtime control flow diimplementasikan pada interpreter melalui instruksi JMP dan JPC. Kedua instruksi ini berfungsi untuk mengubah nilai program counter sehingga eksekusi tidak selalu berjalan lurus ke instruksi berikutnya. Dengan adanya instruksi ini, intermediate code hasil translasi `if`, `if-else`, dan `while` dapat dijalankan sesuai alur kontrol program.

```

void Interpreter::executeInstruction(
...
    case Opcode::JMP:
        validateInstructionTarget(instruction, instructions);
        pc = static_cast<std::size_t>(instruction.operand);
        break;
    case Opcode::JPC: {
        validateInstructionTarget(instruction, instructions);
        RuntimeValue condition = popValue();
        if (condition.integer == 0) {
            pc = static_cast<std::size_t>(instruction.operand);
        }
        break;
    }
...

```

3.8.1 Implementasi Instruksi JMP

Instruksi `JMP` digunakan untuk melakukan lompatan tanpa syarat. Interpreter terlebih dahulu memvalidasi target instruksi, lalu mengubah nilai `pc` langsung menjadi operand dari instruksi tersebut.

```
case Opcode::JMP:
    validateInstructionTarget(instruction, instructions);
    pc = static_cast<std::size_t>(instruction.operand);
    break;
```

3.8.2 Implementasi Instruksi JPC

Instruksi `JPC` digunakan untuk melakukan lompatan bersyarat. Interpreter mengambil nilai kondisi dari stack menggunakan `popValue()`, lalu melompat ke target operand jika nilai kondisi tersebut bernilai 0 atau false.

```
case Opcode::JPC: {
    validateInstructionTarget(instruction, instructions);
    RuntimeValue condition = popValue();
    if (condition.integer == 0) {
        pc = static_cast<std::size_t>(instruction.operand);
    }
    break;
}
```

3.8.3 Implementasi Runtime Flow pada If-Else dan While

Pada runtime, `JPC` digunakan untuk melompati blok ketika kondisi false, sedangkan `JMP` digunakan untuk melewati blok else atau kembali ke awal loop. Karena `pc` sudah dinaikkan sebelum instruksi dieksekusi, target jump cukup diisi langsung dengan `instruction.operand`.

3.9 Implementasi Runtime Error Handling

Runtime error handling ditambahkan agar interpreter tidak melakukan akses stack, memory, atau instruksi secara tidak valid. Pada tahap semantic analysis, program hanya memeriksa kebenaran struktur dan tipe berdasarkan source code. Namun, beberapa kesalahan baru dapat diketahui saat program dijalankan, misalnya stack kosong ketika operasi membutuhkan operand, akses memory ke alamat yang tidak tersedia, pembagian dengan nol, atau jump ke baris instruksi yang tidak valid.

Kesalahan runtime direpresentasikan dengan kelas `RuntimeError`. Kelas ini merupakan turunan dari `std::runtime_error`, sehingga dapat dilempar dari bagian interpreter dan ditangkap

di main. Dengan desain ini, interpreter dapat menghentikan eksekusi ketika menemukan kondisi berbahaya, tetapi program utama tetap dapat menulis hasil yang sudah terbentuk ke file output.

```
class RuntimeError : public std::runtime_error {
public:
    explicit RuntimeError(const std::string& message);
};

RuntimeError::RuntimeError(const std::string& message)
    : std::runtime_error(message) {}
```

Pada main, runtime error tidak langsung membuat keseluruhan program gagal menulis output. Jika ada output yang sudah sempat dihasilkan sebelum error, output tersebut tetap disimpan. Setelah itu, pesan error ditulis pada bagian khusus Runtime Error.

```
try {
    Interpreter interpreter;
    interpreter.execute(generator.getInstructions(),
runtimeStream);
    programOutput = runtimeStream.str();
} catch (const RuntimeError& e) {
    programOutput = runtimeStream.str();
    runtimeError = e.what();
}
```

Pendekatan ini membuat proses pengujian lebih mudah karena pengguna dapat melihat intermediate code, output parsial, dan penyebab runtime error dalam satu file hasil.

3.9.1 Implementasi Validasi Stack

Stack pada interpreter memiliki dua fungsi. Bagian awal stack digunakan sebagai memory frame untuk variabel, sedangkan bagian atas stack digunakan untuk nilai sementara hasil evaluasi ekspresi. Karena dua fungsi ini berada dalam satu vector yang sama, interpreter harus membedakan area memory dan area operand stack.

Stack overflow dicek ketika nilai baru akan dimasukkan ke stack. Kondisi ini terjadi ketika ukuran stack sudah mencapai `maxStackSize`. Batas ini disimpan pada `RuntimeConfig` agar interpreter tidak menggunakan memory tanpa batas ketika program menghasilkan terlalu banyak nilai sementara atau mengalami loop yang terus menambah stack.

Stack underflow dicek ketika interpreter akan mengambil nilai dari stack, tetapi hanya tersisa area memory. Pada kondisi tersebut, operasi seperti `OPR`, `STO`, `JPC`, `WRT`, atau `WRTLN` mencoba mengambil operand yang sebenarnya tidak tersedia.

```

void Interpreter::pushValue(RuntimeValue value) {
    if (stack.size() >= config.maxStackSize) {
        throw RuntimeError(
            "Runtime Error: Stack Overflow (limit " +
            std::to_string(config.maxStackSize) + ") "
        );
    }
    stack.push_back(value);
}

RuntimeValue Interpreter::popValue() {
    if (stack.size() <= memorySize) {
        throw RuntimeError("Runtime Error: Stack Underflow");
    }
    RuntimeValue value = stack.back();
    stack.pop_back();
    return value;
}

```

3.9.2 Implementasi Validasi Memory Address

Validasi memory address dilakukan sebelum operasi baca dan tulis memory. Memory runtime disimpan pada vector `stack`, tetapi hanya index 0 sampai `memorySize - 1` yang dianggap sebagai alamat memory valid. Alamat di luar range tersebut tidak boleh digunakan oleh instruksi `LOD`, `STO`, `PLO`, maupun `PST`.

Fungsi `validateAddress` menjadi satu pintu validasi memory. Dengan cara ini, semua operasi baca dan tulis memory menggunakan aturan validasi yang sama.

```

void Interpreter::validateAddress(int address) const {
    if (address < 0 || static_cast<std::size_t>(address) >=
memorySize) {
        throw RuntimeError(
            "Runtime Error: Memory Access Out of Bounds at address
" +
            std::to_string(address)
        );
    }
}

```

Setelah alamat dinyatakan valid, operasi baca dilakukan oleh `readMemory`, sedangkan operasi tulis dilakukan oleh `writeMemory`.

```

RuntimeValue Interpreter::readMemory(int address) const {
    validateAddress(address);
    return stack[static_cast<std::size_t>(address)];
}

```

```
void Interpreter::writeMemory(int address, RuntimeValue value) {
    validateAddress(address);
    stack[static_cast<std::size_t>(address)] = value;
}
```

3.9.3 Implementasi Validasi Jump Target

Target jump dicek agar tidak menunjuk ke instruksi di luar daftar instruksi. Validasi ini digunakan oleh `JMP`, `JPC`, dan `CAL`. Tanpa validasi target, program counter dapat diarahkan ke index instruksi yang tidak ada, sehingga interpreter akan menjalankan memory yang tidak valid atau berhenti dengan kondisi yang tidak jelas. Pada model interpreter ini, operand dari instruksi jump berisi nomor baris tujuan. Nomor baris tersebut harus berada dalam range 0 sampai `instructions.size() - 1`.

```
void Interpreter::validateInstructionTarget(
    const Instruction& instruction,
    const std::vector<Instruction>& instructions
) const {
    if (instruction.operand < 0 ||
        static_cast<std::size_t>(instruction.operand) >=
        instructions.size()) {
        throw RuntimeError(
            "Runtime Error: Label not found for " +
            CodeGenerator::opcodeToString(instruction.opcode) +
            " target " + std::to_string(instruction.operand)
        );
    }
}
```

Jika target tidak valid, interpreter melempar `RuntimeError` dengan informasi opcode dan target yang bermasalah. Pesan ini membantu debugging karena pengguna dapat mengetahui instruksi jump mana yang gagal.

3.9.4 Implementasi Validasi Operasi Aritmatika

Operasi aritmatika memeriksa overflow dan underflow 32-bit melalui `checkedInt32`. Pemeriksaan ini diperlukan karena nilai runtime integer disimpan dalam `std::int32_t`. Saat operasi dilakukan, perhitungan sementara memakai `std::int64_t` agar hasil yang melebihi batas 32-bit dapat dideteksi sebelum dipaksa kembali menjadi 32-bit.

```
std::int32_t Interpreter::checkedInt32(
    std::int64_t result,
    const std::string& operation
) const {
    if (result > std::numeric_limits<std::int32_t>::max()) {
```

```

        throw RuntimeError(
            "Runtime Error: Overflow during " + operation
        );
    }
    if (result < std::numeric_limits<std::int32_t>::min()) {
        throw RuntimeError(
            "Runtime Error: Underflow during " + operation
        );
    }
    return static_cast<std::int32_t>(result);
}

```

Dengan validasi tersebut, operasi seperti penjumlahan, pengurangan, perkalian, pembagian, modulo, dan negasi tidak langsung melakukan cast ke `int32_t`. Jika hasil lebih besar dari `std::numeric_limits<std::int32_t>::max()`, interpreter menghasilkan Overflow. Jika hasil lebih kecil dari `std::numeric_limits<std::int32_t>::min()`, interpreter menghasilkan Underflow.

Selain batas integer, pembagian dan modulo juga memeriksa nilai pembagi nol. Hal ini dilakukan sebelum operasi pembagian atau modulo dijalankan.

```

if (right.integer == 0) {
    throw RuntimeError("Runtime Error: Division by zero");
}

if (right.integer == 0) {
    throw RuntimeError("Runtime Error: Modulo by zero");
}

```

Operasi aritmatika juga memeriksa kecocokan tipe runtime melalui `resolveRuntimeType`. Jika dua operand memiliki tipe berbeda, interpreter tidak menjalankan operasi dan menghasilkan runtime error.

```

RuntimeType Interpreter::resolveRuntimeType(RuntimeValue val1, RuntimeValue val2) {
    if (val1.type == val2.type) {
        return val1.type;
    } else {
        return RuntimeType::ERROR;
    }
}

```

Validasi tipe ini mencegah operasi yang tidak bermakna, misalnya menjumlahkan integer dengan string. Untuk tipe `REAL`, interpreter menggunakan fungsi khusus `handleFloatingPointOperation` karena nilai real disimpan sebagai pola bit float di dalam integer runtime.

3.10 Implementasi Output Intermediate Code dan Program Output

Output milestone 4 disusun pada program utama setelah proses semantic analysis selesai. Jika tidak terdapat semantic error, program menjalankan CodeGenerator untuk membentuk intermediate code, lalu mencetak daftar instruksi menggunakan `generator.print()`. Setelah itu, daftar instruksi dieksekusi oleh Interpreter, dan hasil eksekusi disimpan sebagai program output.

Jika terjadi runtime error, pesan error tetap dicatat agar dapat ditampilkan pada bagian output akhir.

```
int main(int argc, char* argv[]) {
    ...

    string intermediateCode;
    string programOutput;
    string runtimeError;

    ...

    auto writeSemanticOutput = [&](std::ostream& out) {
        ...

        if (!analyzer.hasErrors()) {
            out << endl;
            out << "=== Intermediate Code ===" << endl;
            out << intermediateCode;

            out << endl;
            out << "=== Program Output ===" << endl;
            out << programOutput;
            if (!programOutput.empty() && programOutput.back() !=
'\n') {
                out << endl;
            }

            if (!runtimeError.empty()) {
                out << endl;
                out << "=== Runtime Error ===" << endl;
                out << runtimeError << endl;
            }
        }
    };

    ...
}
```

BAB 4 Pengujian

4.1 Pengujian Masukkan dan Keluaran Program

Karena output Decorated AST dan Symbol Table yang dihasilkan cukup panjang, tangkapan layar penuh tidak ditampilkan secara keseluruhan pada laporan agar tetap terbaca. Oleh karena itu, laporan hanya menampilkan ringkasan input, nama file output, dan status hasil pengujian. File input, file output lengkap, serta screenshot hasil eksekusi program dapat dilihat pada repository GitHub yang tercantum pada lampiran.

Masukkan	Keluaran
Test 1 - Nama File : input-1.txt {pengujian 1 - bagian tipe data primitif dan operator aritmatika} <pre> program Kalkulator; var x, y : integer; phi : real; inisial : char; pesan : string; hasil : integer; begin (* isi nilai awal untuk setiap variabel *) x := 20; y := 7; phi := 3.14; inisial := 'A'; pesan := 'Halo Arion'; { hitung hasil penjumlahan, pengurangan, dan perkalian } hasil := x + y; hasil := x - y; hasil := x * y; (* pembagian real: 22 dibagi 7 mendekati nilai phi *) phi := 22 / 7; { tampilkan semua hasil ke layar } writeln(hasil); writeln(phi); writeln(inisial); writeln(pesan); end.</pre>	Keluaran File :output-1.txt Seterusnya dapat dilihat di repository pada lampiran

Test 2 - Nama File :input-2.txt (* pengujian 2 - bagian percabangan dan operator logika *) <pre> program CekNilai; var nilai : integer; lulus : integer; begin nilai := 75; lulus := 0; { bandingkan nilai dengan beberapa ambang batas menggunakan operator relasional } if nilai > 90 then writeln('Sangat Baik') else if nilai >= 75 then writeln('Baik') else if nilai < 50 then writeln('Kurang') else if nilai <= 59 then writeln('Cukup') else writeln('Sedang'); (* pastikan nilai tidak sama dengan nol sebelum diproses lebih lanjut *) if nilai <> 0 then writeln('Nilai tidak nol'); { operator NOT membalik nilai logika dari kondisi yang dibungkus kurung } if NOT (nilai < 0) then lulus := 1; (* operator AND mensyaratkan kedua sisi bernilai benar secara bersamaan *) if (nilai >= 75) AND (nilai <= 100) then writeln('Lulus'); { cek apakah nilai di luar rentang yang diperbolehkan } if (nilai < 0) OR (nilai > 100) then writeln('Nilai tidak valid'); (* operator == diuji di sini, perhatikan jika ada bug pada EQL_STATE di lexer *) if nilai == 75 then writeln('Tepat 75'); end.</pre>	Keluaran File :output-2.txt Seterusnya dapat dilihat di repository pada lampiran

Test 3 - Nama File :input-3.txt {pengujian 3 - bagian perulangan dengan for, while, dan repeat} <pre> program Perulangan; var i, n, total : integer; begin n := 5; total := 0; { jumlahkan angka 1 sampai n menggunakan perulangan for naik } for i := 1 to n do begin total := total + i; end; writeln(total); (* cetak angka dari n turun ke 1 menggunakan for downto *) for i := n downto 1 do begin writeln(i); end; { perulangan while akan terus berjalan selama kondisinya benar } i := 1; while i <= n do begin writeln(i); i := i + 1; end; (* repeat dijalankan minimal satu kali sebelum kondisi until diperiksa) i := 1; repeat writeln(i); i := i + 1; until i > n; end.</pre>	Keluaran File :output-3.txt Seterusnya dapat dilihat di repository pada lampiran
Test 4 - Nama File :input-4.txt	Keluaran File :output-4.txt

```
(* pengujian 4 - bagian deklarasi tipe
kompleks, fungsi, dan prosedur *)

program TipeKompleks;
const
    MAX == 10;
    PI  == 3.14159;
type
    Titik == record
        x : real;
        y : real;
    end;
    Daftar == array [1..10] of integer;
var
    p      : Titik;
    data   : Daftar;
    i      : integer;

{ fungsi ini menerima satu bilangan bulat dan
mengembalikan kuadratnya }
function Kuadrat(n : integer) : integer;
begin
    Kuadrat := n * n;
end;

(* prosedur ini mengisi array dengan nilai
kuadrat dari indeksinya *)
procedure IsiArray(arr : Daftar; n : integer);
var
    j : integer;
begin
    for j := 1 to n do
        begin
            arr[j] := j * j;
        end;
    end;
end;

begin
    { isi koordinat titik dengan nilai real }
    p.x := 3.0;
    p.y := 4.0;

    (* panggil prosedur untuk mengisi data,
    lalu tampilkan hasilnya *)
    IsiArray(data, MAX);

    for i := 1 to MAX do
        begin
            writeln(data[i]);
        end;

        writeln(Kuadrat(5));
```

Seterusnya dapat dilihat di repository pada lampiran

end.	
Test 5 - Nama File :input-5.txt {pengujian 5 - bagian case, operator div dan mod, serta variasi komentar} <pre> program KasusLengkap; var angka : integer; bagi : integer; sisa : integer; harakat : char; begin angka := 17; { bagi integer: 17 dibagi 5 hasilnya 3, bukan 3.4 } bagi := angka div 5; (* sisa bagi: 17 mod 5 hasilnya 2 karena 5 masuk 3 kali dengan sisa 2 *) sisa := angka MOD 5; writeln(bagi); writeln(sisa); { gunakan case of untuk memeriksa nilai sisa pembagian satu per satu } case sisa of 0 : writeln('Habis dibagi 5'); 1 : writeln('Sisa 1'); 2 : writeln('Sisa 2'); 3 : writeln('Sisa 3'); 4 : writeln('Sisa 4'); end; (* simpan satu karakter huruf ke dalam variabel bertipe char *) harakat := 'Z'; writeln(harakat); end.</pre>	Keluaran File :output-5.txt Seterusnya dapat dilihat di repository pada lampiran
Test 6 - Nama File :input-6.txt {pengujian 6 - identifier digunakan sebelum dideklarasikan} <pre> program IdentifierBelumDeklarasi; begin</pre>	Keluaran File :output-6.txt Seterusnya dapat dilihat di repository pada lampiran

<pre> nilai := 10; writeln(nilai); end. </pre>	
Test 7 - Nama File :input-7.txt {pengujian 7 - constraint array} program KesalahanArrayRecord; type Daftar == array [1..10] of integer; var data : Daftar; i : integer; begin i := 1; while i <= 10 do begin data[i] := i*i; writeln(data[i]); i := i + 1; end; end	Keluaran File :output-7.txt Seterusnya dapat dilihat di repository pada lampiran
Test 8 - Nama File :input-8.txt {pengujian 8 - constraint record} program CoordinateVertex; type Vertex == record x : integer; y : integer; z : integer; end; var v : Vertex; begin v.x := 1; v.y := 2; v.z := 2; write('('); write(v.x); write(','); write(v.y); write(','); write(v.z); write(')'); end.	Keluaran File :output-8.txt Seterusnya dapat dilihat di repository pada lampiran

--	--

BAB 5

Kesimpulan dan Saran

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian pada Milestone 4, dapat disimpulkan bahwa program Arion telah berhasil dikembangkan hingga tahap intermediate code generation dan interpreter. Program tidak hanya mampu melakukan lexical analysis, syntax analysis, dan semantic analysis, tetapi juga mampu menerjemahkan Decorated AST menjadi intermediate code berbasis stack machine. Intermediate code yang dihasilkan kemudian dapat dieksekusi oleh interpreter melalui mekanisme fetch-decode-execute.

Implementasi code generator berhasil menerjemahkan berbagai konstruksi bahasa Arion, seperti literal, variabel, ekspresi unary, ekspresi biner, assignment statement, write/writeln, serta struktur kontrol seperti if, if-else, dan while. Proses translasi dilakukan dengan pendekatan traversal AST secara rekursif dan menggunakan instruksi virtual machine seperti `LIT`, `LOD`, `STO`, `INT`, `JMP`, `JPC`, `OPR`, dan `RET`. Selain itu, mekanisme backpatching juga digunakan untuk menangani target lompatan pada struktur kontrol yang belum diketahui saat instruksi pertama kali dibangkitkan. Interpreter yang dikembangkan berhasil menjalankan intermediate code dengan menggunakan model stack machine. Interpreter mampu mengelola stack runtime, memori variabel, program counter, serta menjalankan operasi aritmatika, relasional, percabangan, perulangan, dan output program. Selain itu, interpreter juga telah dilengkapi dengan validasi runtime untuk menangani berbagai kemungkinan kesalahan, seperti stack overflow, stack underflow, pembagian dengan nol, akses memori tidak valid, invalid jump address, serta overflow dan underflow aritmatika.

5.2 Saran

Untuk pengembangan selanjutnya, bahasa Arion dapat diperluas agar memiliki fitur bahasa pemrograman yang lebih lengkap dan fleksibel. Salah satu pengembangan yang dapat dilakukan adalah penyempurnaan dukungan terhadap procedure dan function, termasuk penggunaan parameter, nilai kembalian, serta pemanggilan antar subprogram. Dengan adanya fitur tersebut, program Arion dapat ditulis secara lebih modular dan tidak hanya terbatas pada alur program utama. Selain itu, bahasa Arion juga dapat dikembangkan dengan dukungan tipe data yang lebih kompleks, seperti array multidimensi, record yang lebih fleksibel, string operation, serta tipe data bentukan pengguna. Pengembangan ini akan membuat bahasa Arion mampu merepresentasikan struktur data yang lebih beragam sehingga dapat digunakan untuk menyelesaikan permasalahan yang lebih kompleks.

Pengembangan lain yang dapat dilakukan adalah memperkaya struktur kontrol dalam bahasa Arion. Selain percabangan dan perulangan dasar, bahasa ini dapat mendukung bentuk kontrol yang lebih lengkap seperti for loop, repeat-until, case statement, break, dan continue. Dengan adanya struktur kontrol tersebut, penulisan program menjadi lebih ekspresif dan mendekati bahasa pemrograman umum. Bahasa Arion juga dapat ditingkatkan dari sisi input dan output. Saat ini output program telah dapat dilakukan melalui write dan writeln, namun kedepannya dapat ditambahkan mekanisme input dari pengguna, manipulasi string, serta pembacaan dan penulisan file. Fitur ini akan membuat program Arion tidak hanya menghasilkan keluaran statis, tetapi juga dapat berinteraksi dengan pengguna dan data eksternal.

Dengan pengembangan fitur-fitur tersebut, bahasa Arion diharapkan dapat menjadi bahasa pemrograman sederhana yang lebih lengkap, ekspresif, dan mudah digunakan. Pengembangan ini juga dapat menjadi dasar untuk memahami lebih lanjut bagaimana suatu bahasa pemrograman dirancang, diterjemahkan, dan dieksekusi oleh interpreter.

Lampiran

Link Github : <https://github.com/Alpaomega1136/JML-Tubes-IF2224-2026.git>

Link Diagram :

Pembagian Tugas :

NIM	Nama	Pengerjaan pada milestone ini	Persentase
13524015	Mahatma Brahmana	mplementasi control flow (IF-ELSE, WHILE, JMP, JPC), runtime execution, dokumentasi control flow, laporan	25%
13524057	Benedict Darrel Setiawan	Runtime validation dan error handling, overflow/underflow detection, wrapper untuk type checking, array dan record access, dokumentasi pengujian dan validasi runtime, laporan	25%
13524059	Raymond Jonathan D P J	Interpreter core, stack machine, manajemen memori runtime, dokumentasi interpreter, laporan	25%
13524111	Reynard Anderson W	Intermediate code generator, translasi expression dan assignment, dokumentasi code generation, laporan	25%

Daftar Pustaka

Compilers: Principles, Techniques, and Tools. (2006). *Compilers: Principles, techniques, and tools*. Pearson.